

ECO387C - Fall 2024

Discussion 11

Raymond He & Husang Kim

1 Introduction

1.1 Background

Consider the neoclassical growth model in a deterministic environment, where the agent solves the following problem:

$$\max_{\{c_t, k_{t+1}\}_{t=0}^{\infty}} \sum_{t=0}^{\infty} \beta^t u(c_t)$$

subject to the resource constraint:

$$c_t + k_{t+1} = f(k_t) + (1 - \delta)k_t.$$

The Bellman equation for this problem is:

$$v(k) = \max_{\{k'\}} \{u(f(k) + (1 - \delta)k - k') + \beta V(k')\}$$

To solve this problem recursively, we need to find solve for the value function $v(k)$ and the policy function for next period's capital $k' = g(k)$ (finding the consumption policy function becomes trivial once we know the policy function for next period's capital).

We can solve for them via making an initial guess of the appropriate function and iterating on the value function or, if your problem is 'well-behaved', the consumption-Euler equation. Previously, we have implemented this practically by replacing the state space (which usually is continuous with infinitely many elements) with a **discretized** grid. Thus, the function is actually stored on our computers as a vector of function evaluations over the corresponding grid point, and when we invoke the max operator, we're actually picking the largest value among a finite set of function evaluations.

1.2 Motivation

While this approach is extremely robust and relatively simple to implement, there are a few important limitations.

1. **Accuracy:** This approach suffers from accuracy issues and a lot of grid-points are required to ensure that your solution is relatively close to the true solution and not due to rounding error from a coarse grid.
2. **Speed:** On modern computing hardware, creating a fine grid of say, 1000 elements, is not an issue in one dimension but rapidly becomes untenable as the number of dimensions increases to the point of infeasibility. This is known as the **curse of dimensionality**.

To **alleviate** these two issues, we will learn an alternative solution method for numerically solving dynamic optimization problems called **collocation**.¹

2 Collocation

Consider the Bellman equation for the optimal growth model again:

$$v(k) = \max_{\{k'\}} \{u(f(k) + (1 - \delta)k - k') + \beta V(k')\}$$

Suppose that we can write the value function in the following **approximate** form:

$$v(k) \approx \sum_{j=1}^N a_j \phi_j(k)$$

where N refers to the number of **pre-selected** basis functions $\phi_j(k)$ (which itself is a function of the state variable(s)) with corresponding coefficients a_j .

If we substitute this representation into both sides of the Bellman equation:

$$\sum_{j=1}^N a_j \phi_j(k_i) = \max_{\{k'\}} \left[u(f(k_i) + (1 - \delta)k - k') + \beta \sum_{j=1}^N a_j \phi_j(k') \right], i = 1, \dots, N,$$

this becomes a system of N non-linear equations in N unknowns coefficients a_j ! I.e., the idea is to solve for N coefficients at n given **collocation nodes** for k_i for $i = 1 \dots N$. To identify these coefficients, collocation forces this approximate form to be exactly correct

¹To manage expectations, collocation is **NOT** a solution to the curse of dimensionality, nor will it automatically result in speed-gains. However, this framework can **ACCOMMODATE** more advanced tools which would not have been possible in a discrete grid. For more details on sparse grids, see here.

at the aforementioned collocation nodes.²

This means that if you already know that your value function fulfills certain properties such as smoothness, then with an appropriate corresponding choice of the family of basis functions, you can achieve the same level of accuracy as the previous discrete state space method but with fewer nodes and thus fewer required function evaluations!

2.1 Implementation

Let Φ_{ij} denote the elements of the $N \times N$ **collocation matrix**:

$$\Phi_{ij} = \phi_j(k_i), i, j = 1, \dots, N$$

Each row corresponds to the basis functions evaluated at that node. I.e. this is a matrix of every combination of the N basis functions evaluated at the N nodes. The LHS of the the Bellman equation is now this matrix product:

$$\Phi \mathbf{a},$$

while the RHS of the Bellman equation is the vector-valued function $\mathbf{v}(\mathbf{a})$ with typical element:

$$v_i(\mathbf{a}) = \max_{\{k'\}} \left[u(f(k_i) + (1 - \delta)k - k') + \beta \sum_{j=1}^N a_j \phi_j(k') \right]$$

Putting all this together, this problem is now to find a the vector of coefficients $\mathbf{a}$ that solves the following **collocation equation**:

$$\Phi \mathbf{a} = \mathbf{v}(\mathbf{a})$$

Thus, the value function iteration is now guessing an initial vector $\mathbf{a}^0$ and iterating on the coefficients until your pre-specified error tolerance condition is satisfied:

$$\mathbf{a}^{l+1} = \Phi^{-1} \mathbf{v}(\mathbf{a}^l), l = 0, 1, 2, \dots$$

However, one of the biggest advantages of collocation is that we can we can update our guesses much more intelligently than before, meaning that we'll likely find converge in much fewer iterations than before.³ Specifically, we can now compute the gradient, which will tell us how much we need to update our guess by.

²This broader class of methods is also known as **projection** methods. They share the same idea as collocation, but differ in how the number of basis functions relate to the number of nodes and thus, how the coefficients are determined.

³In fact, naively updating the coefficients in this manner is likely to take longer than we had just used a discrete grid! The number of iterations required is likely to stay constant or at best, drop a tiny bit. Meanwhile, each iteration is going to take longer because it involves numerical root-finding over a continuous function, which can be computationally costly.

2.2 Newton's Method

Suppose we want to find the root of a function $f(x)$, i.e. the x^* such that $f(x^*) = 0$. One of the most popular methods is Newton's method:

1. Let x^0 be an arbitrary initial guess.
2. Consider a first-order approximation of $f(x)$ around x^0 :

$$f(x) \approx f(x^0) + f'(x^0)(x - x^0)$$

3. Find x^1 such that $f(x^0) + f'(x^0)(x - x^0) = 0$. I.e.

$$x^1 = x^0 - \frac{f(x^0)}{f'(x^0)}$$

4. Iterate until x^{l+1} is close enough to x^l .

The above is for when x is a scalar, but if $\mathbf{x}$ is a **vector**, then for an initial guess $\mathbf{x}^0$:

$$\mathbf{x}^{l+1} = \mathbf{x}^l - \frac{f(\mathbf{x}^l)}{f'(\mathbf{x}^l)}$$

where $f'(\mathbf{x}^l)$ is the **Jacobian**, or matrix of partial derivatives, of $f(\mathbf{x})$ evaluated at $\mathbf{x}^l$:

$$\frac{\partial f_i(\mathbf{x}^l)}{\partial x_j}, i, j = 1, \dots, N$$

This is useful, since we're trying to find the vector of coefficients $\mathbf{a}$ such that

$$f(\mathbf{a}) \equiv \Phi \mathbf{a} - \mathbf{v}(\mathbf{a}) = \mathbf{0}$$

This implies that we can update $\mathbf{a}$ in the following way:

$$\mathbf{a}^{l+1} = \mathbf{a}^l - \frac{\Phi \mathbf{a}^l - \mathbf{v}(\mathbf{a}^l)}{\Phi - v'(\mathbf{a}^l)}$$

where $v'(\mathbf{a}^l)$ has the following elements:

$$\frac{\partial v_i(\mathbf{a}^l)}{\partial a_j} = \beta \phi_j(k'_i(k_i; \mathbf{a}^l)),$$

and the optimal k' is chosen by solving a maximization problem:

$$k'(k; \mathbf{a}^l) = \arg \max \left[u(f(k_i) + (1 - \delta)k - k') + \beta \sum_{i=1}^N a_j \phi(k') \right]$$

3 Basis Functions

The only other thing that we need to operationalize collocation is which family of basis functions should be used. In numerical analysis, three families of basis functions are frequently used:

1. Piecewise-linear splines
2. Piecewise-cubic splines
3. Chebyshev polynomials

As you hopefully recall from Discussion 2, Linear(cubic) splines are a series of linear(cubic) polynomials spliced together to form a continuous function. Linear splines are best when you don't want to impose curvature, whereas cubic splines are ideal for approximating smooth functions.

The last family of basis functions are **Chebyshev polynomials**. As you may recall from Problem Set 2, we learned that the monomial family of basis functions $(x^0, x^1, x^2, \dots, x^N)$ on an evenly-spaced grid is a terrible idea in practice, because the larger N is, the more wildly the approximated function oscillates around the end-points.

A solution to this issue, or Runge's phenomenon, are **Chebyshev nodes** and the corresponding Chebyshev polynomials. Something that we have taken for granted so far is that our points are linearly spaced across the interval that we want to approximate our function. However, we are free to space our grid points however we want. Chebyshev nodes are spaced out to minimize the worst-case scenario for possible approximation errors. Over an interval $[a, b]$, our N nodes would be distributed in the following way:

$$x_i = \frac{1}{2}(a+b) + \frac{1}{2}(b-a)\cos\left(\frac{i}{N}\pi\right), i = 0, 1, \dots, N$$

Chebyshev polynomials are a special sequence of **orthogonal polynomials** defined by the following recursion:

$$T_0(z) = 1$$

$$T_1(z) = z$$

$$T_2(z) = 2z^2 - 1$$

$$T_j(z) = 2zT_{j-1} - T_{j-2}(z), j = 2, 3, \dots$$

where $z = 2\frac{x-a}{b-a} - 1$. These polynomials are specifically rigged so that their roots are the corresponding Chebyshev nodes.